Matrix-Free Methods for Long-Only Portfolio Optimization

Thomas Schmelzer[1], Martin Stoll[2], and Michael Wolf[3,4]

[1]Jebel Quant Research, Abu Dhabi
[2]Faculty of Mathematics, TU Chemnitz
[3]Department of Economics, University of Zurich
[4]ADIA Lab, Abu Dhabi

July 2, 2026

Abstract

The long-only mean-variance portfolio – minimising variance for a given return target subject to a budget constraint and non-negative weights – is routinely solved by assembling an $n \times n$ sample covariance matrix and dispatching it to a general-purpose quadratic program. Both steps are unnecessary: the covariance matrix is not a primitive of the problem. It is a computational detour.

The detour is avoidable. The portfolio weights require only $v = \hat{\Sigma}^{-1}\mathbf{1}$; conjugate gradients computes this matrix-free, evaluating each product $\hat{\Sigma}\,v$ as two $O(nT)$ passes over X without ever forming $\hat{\Sigma} = X^\top X/T$. Non-negativity is enforced by a primal-dual active-set outer loop that alternately removes assets with negative weight (primal step) and re-adds assets that violate dual feasibility (dual step), restarting the inner solver on the modified active set each time. The mean-variance problem with a return tilt ρ reduces to two such solves with the budget multiplier recovered analytically. The $O(\sqrt{\kappa})$ iteration count of CG, where $\kappa = \kappa(\hat{\Sigma})$ is the spectral condition number, contrasts sharply with the $O(\kappa)$ rate of projected-gradient methods that do not exploit Krylov structure.

Ledoit-Wolf shrinkage [18], already standard practice for statistical reasons, also compresses κ and reduces solver iterations as a free by-product. The effect is modest at the statistically optimal intensity ($\alpha_{\mathrm{LW}} \approx 0.017$), because the Frobenius objective is insensitive to the near-zero eigenvalues that drive ill-conditioning. At the heavier intensities practitioners apply for out-of-sample stability ($\alpha \approx 0.3$–0.5) the conditioning benefit is decisive and requires no additional preconditioning step. That practitioners applying heavy shrinkage are simultaneously conditioning their solver has not been made explicit in the portfolio optimisation literature. RMT eigenvalue cleaning [16, 3] provides a more decisive resolution, decoupling the two objectives; the Woodbury direct solver for the RMT target is developed in the companion paper [26].

We benchmark against general-purpose QP solvers (Clarabel interior-point, OSQP operator-splitting) and a non-Krylov first-order baseline. On S&P 500 equity data CG requires 684 iterations without shrinkage, falling to 82 under heavy LW shrinkage ($\alpha = 0.5$). Called head-to-head against the same solvers through their native APIs the algorithmic gain is roughly an order of magnitude; against those solvers wrapped in a general modelling layer the end-to-end gap is larger still, as it also absorbs problem-construction overhead. CG's runtime grows substantially subquadratically in n while interior-point methods scale as $O(n^3)$ per iteration, and warm-starting across the return-tilt sweep traces a 50-point long-only efficient frontier orders of magnitude faster than a naïve loop over a modelling layer. A rolling-window out-of-sample study on the same universe shows that the heavy-shrinkage regime which makes the solver fast is also the regime that minimises realised out-of-sample variance while lowering turnover: the statistical and numerical objectives, far from conflicting, are reconciled at the intensities practitioners already use.

1 Introduction

The long-only minimum variance portfolio allocates capital across n assets to minimize portfolio return variance subject to two constraints: the weights must sum to one (the portfolio is fully invested) and must be non-negative. The only input required in theory is the $n \times n$ covariance matrix Σ of the $n \times 1$ vector of asset returns. In practice, Σ is unknown and must be estimated from historical data. To this end, one uses a $T \times n$ matrix of demeaned returns X, where T denotes the length of the estimation window. The classical estimator of Σ is the sample covariance matrix $\hat{\Sigma} = X^\top X/T$.

The Lagrangian for the problem

$$\min_w \; w^\top \hat{\Sigma} w \quad \text{subject to} \quad \mathbf{1}^\top w = 1,$$

where $\mathbf{1}$ denotes a conformable vector of ones, is

$$\mathcal{L}(w, \lambda) = w^\top \hat{\Sigma} w - \lambda(\mathbf{1}^\top w - 1),$$

whose stationarity condition $2\hat{\Sigma}w = \lambda \mathbf{1}$ immediately gives $w \propto \hat{\Sigma}^{-1}\mathbf{1}$; this was made explicit by [21] in the context of the efficient frontier. The budget constraint is recovered by the single rescaling

$$w = v/(\mathbf{1}^\top v), \qquad v = \hat{\Sigma}^{-1}\mathbf{1}.$$

What has *not* been exploited is this structure computationally: the standard pipeline still forms $\hat{\Sigma} = X^\top X/T$ explicitly before solving.

This paper makes two contributions. First, we apply conjugate gradients (CG) to $\hat{\Sigma}v = \mathbf{1}$ *matrix-free*: $\hat{\Sigma} = X^\top X/T$ is never assembled. Each CG iteration requires only two matrix-vector products with X, reducing per-iteration cost from $O(n^2)$ to $O(nT)$ and eliminating $O(n^2)$ storage (the active-set loop further reduces this to $O(kT)$, $k \leq n$; see Table 2). Non-negative weights are enforced by a primal-dual active-set loop that alternately removes assets with negative weight and re-adds assets violating dual feasibility, calling the CG solver on the modified active set at each step. Because CG is a Krylov method, it builds successive iterates in the Krylov subspace $\mathcal{K}_k(\hat{\Sigma}, \mathbf{1})$ and achieves $O(\sqrt{\kappa})$ convergence, where $\kappa = \kappa(\hat{\Sigma})$ is the spectral condition number (Proposition 6.1). This $\sqrt{\kappa}$ rate is the defining advantage of Krylov methods over general first-order schemes, which require $O(\kappa)$ iterations without exploiting the subspace structure.

Second, we show that Ledoit-Wolf shrinkage [18], applied at the heavier intensities practitioners already use for out-of-sample stability, also compresses the eigenvalue spread of the system matrix and accelerates CG convergence (Proposition 5.1) as a free by-product, with no additional preconditioning step. On S&P 500 equity data, heavy LW shrinkage ($\alpha = 0.5$) reduces CG iterations from 684 to 82; the Frobenius-optimal intensity $\alpha_{\text{LW}} \approx 0.017$ has a more modest effect (516 iterations), because the Frobenius loss is insensitive to the near-zero eigenvalues that drive ill-conditioning. The Frobenius surrogate and the conditioning number are therefore minimised at different α, which might suggest a genuine statistics-versus-speed trade-off. A rolling-window out-of-sample study (Section 7.3) shows there is none: realised out-of-sample variance is minimised at $\alpha \approx 0.3$, inside the heavy-shrinkage range that also gives the conditioning benefit, while turnover falls monotonically in α. The intensity that makes the solver fast is thus also the intensity a risk-conscious practitioner would choose, and the apparent tension is an artefact of the Frobenius surrogate rather than a property of the portfolio problem. The observation that heavy shrinkage conditions the solver as a by-product does not appear to have been made explicit in the portfolio optimisation literature. RMT eigenvalue cleaning [16, 3] decouples the two objectives more decisively; the Woodbury direct solver for the RMT target is developed in the companion paper [26].

CG of Hestenes and Stiefel [12] is a cornerstone of large-scale scientific computing; see, e.g., Golub and Van Loan [9]. To the best of our knowledge it has not been applied to portfolio optimisation in the matrix-free form described here. The closest related work applies GMRES to the minimum variance problem when $\hat{\Sigma}$ is singular [1]; that formulation still assembles $X^\top X$ and does not exploit the SPD structure or consider preconditioning. General-purpose solvers such as Clarabel [10] and OSQP [28] handle the long-only constraint natively but require an assembled covariance matrix as input. Markowitz's Critical Line Algorithm [20] solves the full parametric efficient frontier problem via simplex-like pivots ($O(n^2)$ per pivot, $O(n^3)$ total in the worst case); a clean open-source implementation is available in [25]. The CLA targets the entire analytical frontier rather than a fixed grid of solves, and does not exploit the matrix-free structure of $\hat{\Sigma} = X^\top X/T$.

Computing a discrete efficient frontier via warm-started iterative solves has not, to the authors' knowledge, been studied as a method in its own right. The CLA [20, 25] traces the exact analytical frontier but scales as $O(n^3)$ in the worst case; a naïve grid of independent solver calls does not exploit continuity in the return target ρ. The warm-start described in Section 7 transfers the active-set mask and weight vector between adjacent ρ points, so each CG restart begins near the optimum and the total sweep costs a small multiple of a single solve. Parametric warm-starting of active-set QP solvers is analysed in the numerical optimisation literature [24]; combining it with a matrix-free Krylov inner solver on a changing active set is new.

Three features of the problem make the matrix-free approach non-obvious in standard QP form. First, all prior solvers treat the assembled $\hat{\Sigma}$ as a required input; recognising that $v = \hat{\Sigma}^{-1}\mathbf{1}$ is the only quantity needed requires stepping outside the QP abstraction. Second, recovering the budget multiplier analytically – reducing the long-only problem to a single SPD linear solve rather than a constrained QP – exploits the rank-one structure of the equality constraint, which is not surfaced by general-purpose QP interfaces. Third, restarting a Krylov inner solver on a changing active-asset subset each outer step, without re-initialising unnecessarily, is a non-trivial integration of the active-set and iterative-solver layers.

The scope is deliberately the pure long-only minimum-variance and mean-variance problem: the matrix-free reduction exploits its specific structure – a single SPD solve with a rank-one budget constraint – and richer constraint sets such as sector caps, turnover limits, or factor-exposure bounds fall outside it and are better served by a general-purpose QP solver (Section 6).

For completeness, Section 4 also includes a simplex-projection first-order comparator that handles both constraints simultaneously without an outer active-set loop.

The remainder of the paper is organized as follows. Sections 2–3 develop the problem formulation and non-negativity strategies. Section 5 covers shrinkage targets and preconditioning: the scaled-identity LW estimator and factor-model targets (companion paper [26] covers RMT eigenvalue cleaning). Section 6 presents the concrete solver implementations and their asymptotic complexity. Section 7 reports empirical results on S&P 500 and FTSE 100 data, a scaling study, and an efficient frontier sweep, benchmarking against Clarabel, OSQP, and a first-order comparator.

2 Problem Formulation

The long-only mean-variance portfolio solves

$$\min_{w} \; w^\top \Sigma\, w - \rho\, \mu^\top w \quad \text{subject to} \quad \mathbf{1}^\top w = 1, \quad w \geq 0\ ,$$

where Σ is the $n \times n$ covariance matrix of returns, μ is the $n \times 1$ vector of expected returns, and $\rho \geq 0$ controls the risk-return trade-off; setting $\rho = 0$ recovers the pure long-only minimum variance portfolio. Since both Σ and μ are unknown, in practice they must be replaced by estimates. Let

$X \in \mathbb{R}^{T\times n}$ denote the demeaned return matrix for n assets over the previous T trading periods. The feasible problem formulation then is

$$\min_{w}\ w^\top \hat{\Sigma}\, w - \rho\, \hat{\mu}^\top w \quad \text{subject to} \quad \mathbf{1}^\top w = 1, \quad w \geq 0\ .$$

The classical choice for estimating Σ is the sample covariance matrix $\hat{\Sigma} = X^\top X/T$, for which $w^\top \hat{\Sigma}\, w = \|Xw\|^2/T$, and the problem becomes

$$\min_{w}\ \frac{\|Xw\|^2}{T} - \rho\, \hat{\mu}^\top w \quad \text{subject to} \quad \mathbf{1}^\top w = 1, \quad w \geq 0\ .$$

This reframing is key: the variance term is now a normalised squared norm of Xw, and the gradient $w \mapsto X^\top(Xw)$ can be evaluated with two matrix-vector products, without ever forming $X^\top X$.

Parameter	Type	Description
X	$\mathbb{R}^{T\times n}$	Demeaned return matrix
ρ	$\mathbb{R}_{\geq 0}$	Return-tilt weight (default 0)
μ	$\mathbb{R}^n$	Expected returns (default $\mathbf{0}$)

Table 1: Parameters of the `Problem` specification.

Dual problem Including multipliers for both constraints, the full Lagrangian is

$$\mathcal{L}(w,\, \lambda,\, \nu) = \frac{\|Xw\|^2}{T} - \rho\hat{\mu}^\top w - \lambda(\mathbf{1}^\top w - 1) - \nu^\top w\ ,$$

where $\lambda \in \mathbb{R}$ is the budget multiplier and $\nu \geq \mathbf{0}$ is the vector of non-negativity multipliers. The stationarity condition

$$\tfrac{2}{T}\, X^\top X w = \lambda\mathbf{1} + \nu + \rho\hat{\mu}$$

gives $w^* = \frac{T}{2}(X^\top X)^{-1}s$ with $s = \lambda\mathbf{1} + \nu + \rho\hat{\mu}$. Substituting into $\mathcal{L}$:

$$\mathcal{L}(w^*,\, \lambda,\, \nu) = \tfrac{1}{T}(w^*)^\top X^\top X w^* - s^\top w^* + \lambda\ .$$

From stationarity $\frac{1}{T}X^\top X w^* = \frac{s}{2}$, so the first term equals $\frac{1}{2}s^\top w^*$, giving

$$g(\lambda,\, \nu) = \tfrac{1}{2}s^\top w^* - s^\top w^* + \lambda = -\tfrac{1}{2}s^\top w^* + \lambda\ .$$

Substituting $w^* = \frac{T}{2}(X^\top X)^{-1}s$ yields the dual function

$$g(\lambda,\, \nu) = \inf_{w}\ \mathcal{L}(w,\, \lambda,\, \nu) = \lambda - \tfrac{T}{4}\,(\lambda\mathbf{1} + \nu + \rho\hat{\mu})^\top (X^\top X)^{-1}(\lambda\mathbf{1} + \nu + \rho\hat{\mu})\ ,$$

defined when X has full column rank; the dual problem is $\max_{\lambda,\,\nu\geq\mathbf{0}}\ g(\lambda,\, \nu)$. The feasible set $\{w : \mathbf{1}^\top w = 1,\, w \geq \mathbf{0}\}$ is compact and $w = \frac{1}{n}\mathbf{1}$ is strictly feasible, so Slater's condition holds and strong duality obtains.

At the optimum, complementary slackness ($\nu_i w_i = 0$ for all i) partitions assets into two types. *Active* assets ($w_i > 0$) have $\nu_i = 0$; stationarity then gives

$$\tfrac{2}{T}\,(X^\top X w)_i = \lambda + \rho\hat{\mu}_i\ ,$$

equating each active asset's marginal variance to the budget shadow price adjusted for its expected return. *Excluded* assets ($w_i = 0$) satisfy

$$\nu_i = \tfrac{2}{T}\,(X^\top X w)_i - \lambda - \rho\hat{\mu}_i \geq 0\ .$$

Their marginal variance exceeds this threshold, so holding them at a positive weight would increase variance net of return. The primal-dual loop in Section 4 is a direct implementation of these conditions: it terminates precisely when primal and dual feasibility both hold, which (together with stationarity from each inner solve) is sufficient for global optimality.

3 Positive-Definite Reduction

Ignoring the inequality constraints for now, the Lagrangian for the equality-constrained problem is:

$$\mathcal{L}(w, \lambda) = \frac{\|Xw\|^2}{T} - \rho\hat{\mu}^\top w - \lambda(\mathbf{1}^\top w - 1)\,.$$

Setting the derivative to zero gives the stationarity condition $\frac{2}{T}\hat{\Sigma}\,w = \lambda\mathbf{1} + \rho\hat{\mu}$, where $\hat{\Sigma} = X^\top X/T$. For $\rho = 0$, this reduces to $\frac{2}{T}\hat{\Sigma}\,w = \lambda\mathbf{1}$, determining the solution up to a scalar.

When $\rho = 0$ (pure long-only minimum variance), stationarity gives $2\hat{\Sigma}\,w = \lambda\mathbf{1}$, so w is a scalar multiple of $\hat{\Sigma}^{-1}\mathbf{1}$. Setting $v_1 = \hat{\Sigma}^{-1}\mathbf{1}$ and writing $w = c\,v_1$, the budget constraint $\mathbf{1}^\top w = 1$ determines c:

$$c = \frac{1}{\mathbf{1}^\top v_1} \quad \text{and} \quad w = \frac{v_1}{\mathbf{1}^\top v_1}\ .$$

When $\rho \neq 0$, stationarity gives $2\hat{\Sigma}\,w = \lambda\mathbf{1} + \rho\hat{\mu}$, so $w = \frac{\lambda}{2}\,\hat{\Sigma}^{-1}\mathbf{1} + \frac{\rho}{2}\,\hat{\Sigma}^{-1}\hat{\mu}$. Setting $v_1 = \hat{\Sigma}^{-1}\mathbf{1}$ and $v_2 = \hat{\Sigma}^{-1}\hat{\mu}$ (two SPD solves), the budget constraint determines the multiplier

$$\lambda = \frac{2 - \rho\,(\mathbf{1}^\top v_2)}{\mathbf{1}^\top v_1}\ ,$$

and the weights follow as $w = \frac{\lambda}{2}\,v_1 + \frac{\rho}{2}\,v_2$. In both cases the $(n+1)\times(n+1)$ saddle-point system is never needed and λ is recovered analytically rather than as part of the solve. The long-only problem therefore reduces to solving a symmetric positive definite system $\hat{\Sigma}\,v = b$ for one or two right-hand sides, subject to the remaining inequality constraint $w \geq 0$; the latter is addressed in Section 4, which presents two strategies for doing so. Section 5 shows how Ledoit-Wolf shrinkage replaces $\hat{\Sigma}$ with a better-conditioned estimator $\hat{\Sigma}_{\text{LW}}$ without changing this reduction.

4 Enforcing Non-Negativity

Section 3 reduces the long-only problem to solving an SPD system, leaving $w \geq 0$ as the sole remaining constraint. Two strategies are available.

4.1 Simplex-projection strategy

The first strategy treats the two constraints as a single geometric object: the probability simplex $\Delta_n = \{w \in \mathbb{R}^n : \mathbf{1}^\top w = 1,\ w \geq 0\}$. Rather than separating budget and non-negativity, gradient steps of the form

$$w_{k+1} = \Pi_{\Delta_n}(w_k - \tau\,\nabla f(w_k))$$

project directly onto Δ_n after each descent step, where Π_{Δ_n} is the Euclidean projection. Duchi et al. [8] and Condat [5] show that Π_{Δ_n} can be computed in $O(n \log n)$ by a single sort-and-threshold pass, making each iteration cheap. There is no outer loop: both constraints are handled simultaneously, and the method terminates when the step size $\|w_{k+1} - w_k\|$ falls below a tolerance.

Although $\hat{\Sigma} = \tilde{X}^\top \tilde{X}$ admits a stacked-matrix representation (Section 5), assembling or storing $\tilde{X}$ is unnecessary. The gradient decomposes additively as

$$\nabla f(w) = \hat{\Sigma}\, w = \frac{1-\alpha}{T}\, X^\top (Xw) + \alpha\, \hat{\Sigma}_0\, w\,,$$

and each term is applied separately: the data contribution costs $O(nT)$ via two matrix-vector products with X, and the target contribution costs $O(n^2)$ for a general $\hat{\Sigma}_0$ or $O(n)$ for a scaled identity. The Lipschitz constant $L = \lambda_{\max}(\hat{\Sigma})$, needed for the step size $\tau = L^{-1}$, is estimated once by power iteration on the same two-term operator, also matrix-free.

Algorithm 1 Simplex-projection solver (proximal gradient, matrix-free)

Require: Returns matrix $X \in \mathbb{R}^{T \times n}$; shrinkage intensity α; target $\hat{\Sigma}_0 \in \mathbb{R}^{n \times n}$; tolerance $\varepsilon = 10^{-6}$
Ensure: Weight vector $w \in \Delta_n$

Define operator $\mathcal{A}$: $v \mapsto \frac{1-\alpha}{T} X^\top (Xv) + \alpha\, \hat{\Sigma}_0 v$ *($\hat{\Sigma}$ applied matrix-free)*
Estimate $L \leftarrow \lambda_{\max}(\mathcal{A})$ by power iteration *($O(n_{\text{pow}} \cdot nT)$)*
$\tau \leftarrow L^{-1}$; initialise $w_0 \in \mathbb{R}^n$ randomly; $k \leftarrow 0$
loop
  $g \leftarrow \mathcal{A}(w_k)$ *(gradient, $O(nT)$)*
  $w_{k+1} \leftarrow \Pi_{\Delta_n}(w_k - \tau\, g)$ *(simplex projection, $O(n \log n)$)*
  if $\|w_{k+1} - w_k\|_2 \le \varepsilon$ **then break**
  end if
  $k \leftarrow k + 1$
end loop
return w_{k+1}

4.2 Active-set strategy

The second strategy separates the budget constraint from the non-negativity constraint. The budget-only problem is solved over a dynamically adjusted *active set* of assets; non-negativity is then enforced iteratively without ever modifying the inner solve.

In the *primal step*, any asset with a negative weight is dropped from the active set and the budget-constrained SPD system is re-solved on the reduced universe. Once all active weights are non-negative, the *dual step* checks the KKT gradient condition for every excluded asset: global optimality requires every excluded asset i to satisfy

$$\nabla_i f(w) \;\ge\; \lambda\,, \qquad \nabla_i f(w) = \tfrac{2}{T}(X^\top X w)_i - \rho \hat{\mu}_i\,,$$

where λ is the budget Lagrange multiplier recovered analytically from Section 3. An excluded asset that violates this condition would decrease portfolio variance if held at a small positive weight; it is re-added to the active set and the loop repeats. The loop terminates when both primal and dual feasibility hold; together with stationarity from the inner solve this certifies global optimality.

These two conditions are precisely the complementarity system for the long-only KKT conditions: with the constraint multiplier $s_i = \nabla_i f(w) - \lambda$, an optimum requires $w \ge 0$, $s \ge 0$, and $w_i s_i = 0$.

The active set $\mathcal{A}$ plays the role of the index set on which w is free and $s = 0$; primal violators are active assets with $w_i < 0$ and dual violators are excluded assets with $s_i < 0$. Toggling assets between $\mathcal{A}$ and its complement is therefore a sequence of *principal pivots* on this linear complementarity problem (LCP), and dropping or adding several assets in one step is a *block* principal pivot [14, 15]. Because $\hat{\Sigma}_{\mathrm{LW},\mathcal{A}} \succ 0$ for every $\mathcal{A}$ (Section 3), the governing matrix is a P-matrix, so every principal submatrix is invertible and each pivot is well defined [22].

Block pivots are fast but, like all-at-once exchanges in general, can cycle: a batch drop can over-shoot the support and a later batch add can restore it, returning to an active set already visited. Algorithm 2 therefore runs the batch exchange as a *fast path* guarded by an anti-cycling counter. Let N be the total number of primal and dual violators at the current solve and $\bar{N}$ the fewest seen so far. A batch step is taken whenever it makes progress ($N < \bar{N}$) or a fixed patience budget p has not been exhausted; otherwise the algorithm falls back to a single *Bland-style* pivot that toggles the lowest-indexed violating asset. The least-index single-pivot rule is Murty's principal pivoting method, which cannot cycle [22]; it is what the finite-termination guarantee below actually rests on, with the batch path serving only to reach the optimum faster when it does not stall. This is the anti-cycling construction of Júdice and Pires for block principal pivoting [14], adapted to the budget-constrained minimum-variance LCP.

Algorithm 2 Active-set outer loop (wraps any inner solver f)

Require: Inner solver f; tolerance $\varepsilon = 10^{-6}$; $\alpha > 0$ *(ensures $\hat{\Sigma}_{\mathrm{LW},\mathcal{A}}$ non-singular)*; patience $p_{\max} \in \mathbb{N}$ *(default 3)*

Ensure: Weight vector $w \in \mathbb{R}^n$; outer step count s; total inner iteration count k *(equals s for direct solvers using the $k_{\mathrm{step}} = 1$ sentinel)*

active $\leftarrow \{1, \ldots, n\}$; $s \leftarrow 0$; $k \leftarrow 0$; $\bar{N} \leftarrow n + 1$; $p \leftarrow p_{\max}$ *($\bar{N}$: fewest infeasibilities seen; p: batch-step budget)*
loop
  $(w_{\mathrm{a}}, k_{\mathrm{step}}) \leftarrow f(\text{active})$; $s \leftarrow s + 1$; $k \leftarrow k + k_{\mathrm{step}}$ *($k_{\mathrm{step}} = 1$ sentinel for direct solvers; iterative count for CG)*
  $w_i \leftarrow w_{a,i}$ for $i \in$ active; $w_i \leftarrow 0$ for $i \notin$ active
  $g_i \leftarrow \frac{2}{T}(X^\top X w)_i - \rho\hat{\mu}_i$; $\lambda \leftarrow \mathrm{median}_{j \in \text{active}}(g_j)$ *(stationarity: $g_j \approx \lambda$ for $j \in$ active to solver tolerance)*
  $\mathcal{D} \leftarrow \{i \in \text{active} : w_i < -\varepsilon\}$; $\mathcal{V} \leftarrow \{i \notin \text{active} : g_i < \lambda - \varepsilon\}$ *(primal and dual violators)*
  $\mathcal{H} \leftarrow \mathcal{D} \cup \mathcal{V}$; $N \leftarrow |\mathcal{H}|$
  if $N = 0$ **then**
    break *(KKT satisfied; globally optimal)*
  else if $N < \bar{N}$ **or** $p > 0$ **then** *(fast path: progress, or patience remains)*
    if $N < \bar{N}$ **then** $\bar{N} \leftarrow N$; $p \leftarrow p_{\max}$ **else** $p \leftarrow p - 1$
    active $\leftarrow (\text{active} \setminus \mathcal{D}) \cup \mathcal{V}$ *(batch exchange: drop all $\mathcal{D}$, add all $\mathcal{V}$)*
  else *(anti-cycling fallback: p exhausted)*
    $i^\star \leftarrow \min\{i : i \in \mathcal{H}\}$ *(Bland least-index rule)*
    if $i^\star \in \mathcal{D}$ **then** active $\leftarrow$ active $\setminus \{i^\star\}$ **else** active $\leftarrow$ active $\cup \{i^\star\}$ *(single principal pivot)*
  end if
end loop
return w, s, k

Budget multiplier The λ-computation step of Algorithm 2 recovers the budget shadow price as the median of the active gradients. In exact arithmetic, the inner solver minimises f over $\{w_{\mathcal{A}} : \mathbf{1}^\top w_{\mathcal{A}} = 1\}$, whose stationarity condition is $\frac{2}{T}(X_{\mathcal{A}}^\top X_{\mathcal{A}}\, w_{\mathcal{A}})_j = \lambda$ for every $j \in \mathcal{A}$, so all active gradients equal λ. In practice CG terminates at residual tolerance $\varepsilon = 10^{-5}$, so the active gradients agree with λ up to this tolerance; the median (used in the implementation for robustness; the mean gives the same value in exact arithmetic) recovers λ to the same tolerance.

Theorem 4.1 (Finite convergence of Algorithm 2)**.** *Let $f(w) = w^\top \hat{\Sigma}_{\mathrm{LW}}\, w - \rho \hat{\mu}^\top w$ with $\hat{\Sigma}_{\mathrm{LW}} \succ 0$ (ensured by $\alpha > 0$). Then for any patience budget $p_{\max} \in \mathbb{N}$, Algorithm 2 terminates in finitely many outer iterations and returns the unique global minimizer of $\min_w f(w)$ subject to $\mathbf{1}^\top w = 1,\ w \geq 0$. No non-degeneracy assumption is required.*

Proof. Step 1 (well-posedness of each inner solve). Because $\hat{\Sigma}_{\mathrm{LW}} = (1-\alpha)\hat{\Sigma} + \alpha T_0 \succ 0$, every principal submatrix $\hat{\Sigma}_{\mathrm{LW},\mathcal{A}}$ is SPD, so $\hat{\Sigma}_{\mathrm{LW}}$ is a P-matrix [22]. The equality-constrained problem on active set $\mathcal{A}$, $\min_{w_{\mathcal{A}}:\mathbf{1}^\top w_{\mathcal{A}}=1} f_{\mathcal{A}}(w_{\mathcal{A}})$, has a unique minimiser; CG converges to it (Proposition 6.1).

Step 2 (termination implies global optimality). Stationarity of $f_{\mathcal{A}}$ gives $g_j = \lambda$ for all $j \in \mathcal{A}$. Algorithm 2 exits only when $N = 0$, i.e. (i) all active weights are non-negative and (ii) $g_i \geq \lambda$ for every excluded $i \notin \mathcal{A}$. With the multiplier $s_i = g_i - \lambda$ these are exactly $w \geq 0$, $s \geq 0$, $w_i s_i = 0$: the KKT conditions for the long-only problem. Strict convexity of f makes them sufficient for global optimality. It therefore suffices to show the loop cannot run forever.

Step 3 (the fallback cannot cycle). Index the outer iterates and call a maximal block of consecutive iterates that take the **else** branch (the single Bland pivot) a *fallback run*. Within a run $\bar{N}$ is constant, and each iterate performs one principal pivot on the lowest-indexed violating asset. This is precisely Murty's least-index principal pivoting method for the P-matrix LCP of Step 2, which generates a sequence of complementary bases (here, active sets) in which no basis recurs and which reaches a solution in finitely many pivots [22]. A fallback run is an initial segment of that sequence started from the active set left by the preceding step, so each run is finite, regardless of where it starts.

Step 4 (only finitely many batch steps and runs). The counter N satisfies $0 \leq N \leq n$, and $\bar{N}$ is non-increasing and strictly decreases exactly when the progress branch ($N < \bar{N}$) fires. Hence the progress branch fires at most $n+1$ times. Each remaining fast-path step has $N \geq \bar{N}$ and decrements p; since p is reset to $p_{\max}$ only on a progress step, at most $p_{\max}$ such steps occur between progress events. The number of fallback runs is likewise bounded: a run ends either at termination ($N = 0$) or at a progress step, and there are at most $n+1$ progress steps. Combining, the algorithm takes at most $(n+1) + (n+2)\, p_{\max}$ fast-path steps and at most $n+2$ fallback runs.

Step 5 (finite termination). By Step 3 each of the at most $n+2$ fallback runs is finite, and by Step 4 the fast-path steps are finite in number. Their sum is finite, so the loop reaches $N = 0$ after finitely many outer iterations; Step 2 then certifies the returned w as the unique global minimiser. A crude ceiling is the number of complementary bases, 2^n. □

The 2^n ceiling of Step 5 establishes *finiteness* only; it is not an efficiency estimate, and we make no claim that the worst case is approached. The guarantee is unconditional: the least-index fallback removes the non-degeneracy hypothesis that earlier active-set arguments need, and it is the fallback – not the batch exchange – that the proof rests on. In practice the outer loop never reaches the fallback: across both equity universes and all shrinkage levels reported here it terminates in s between 6 and 18 outer steps (Tables 3–4), with the active set changing by only a few assets per step and the patience budget p never exhausted, so the batch fast path alone certifies optimality and no cycling was observed in any run.

Remark 4.1 (Exact inner solver assumption)**.** *Theorem 4.1 assumes each inner call to $f(\mathcal{A})$ returns the exact minimiser of $f_{\mathcal{A}}$. In the algorithm, CG terminates at relative residual tolerance $\varepsilon = 10^{-5}$,*

so the returned iterate is not the exact minimiser. The practical implication is small: the KKT dual-feasibility check in Algorithm 2 uses the same tolerance ε, so an inexact iterate that satisfies the check to tolerance ε is accepted as optimal. The finite-termination argument of Steps 3–5 holds exactly in exact arithmetic and to working accuracy in finite precision; no cycling was observed in any reported experiment.

Remark 4.2 (Why the fallback is rarely triggered)**.** *The fallback engages only when the batch exchange stalls – when the infeasibility count N fails to improve on its best value $\bar{N}$ for $p_{\max}$ consecutive steps. Such stalls are tied to degenerate configurations of X, each of which corresponds to a polynomial equation in its entries. A batch step fails to reduce N only when a dropped asset is immediately re-added (or vice versa), which requires $g_i = \lambda$ exactly for some toggled i, a codimension-one condition; the related event $\hat{\Sigma}_{\mathrm{LW},\mathcal{A}}^{-1}\mathbf{1} \geq 0$ component-wise at a non-terminal iterate is likewise codimension-one. Both fail almost surely under any distribution for X absolutely continuous with respect to Lebesgue measure; for continuous factor models or i.i.d. noise distributions this holds by construction. Under strictly positive shrinkage ($\alpha > 0$) the minimiser $w^\star(\mathcal{A}) = \hat{\Sigma}_{\mathrm{LW},\mathcal{A}}^{-1}\mathbf{1}/(\mathbf{1}^\top\hat{\Sigma}_{\mathrm{LW},\mathcal{A}}^{-1}\mathbf{1})$ is a rational function of X, further constraining the degenerate set, and on real market data floating-point rounding provides an additional generic-position guarantee. Theorem 4.1 no longer* assumes *these conditions – the fallback guarantees termination unconditionally – but they explain why p was never exhausted in any experiment reported here, so the batch fast path alone always sufficed.*

Warm-starting Two levels of warm-starting are available for Algorithm 2.

Within-solve warm-starting (between outer iterations of a single solve). When the active set changes size, the previous CG solution lives in a different subspace and cannot be reused directly. One can restrict the previous full weight vector $w^{(k)} \in \mathbb{R}^n$ to the new active set $\mathcal{A}^{(k+1)}$ and renormalise:

$$x_0 = \frac{w^{(k)}_{\mathcal{A}^{(k+1)}}}{\mathbf{1}^\top w^{(k)}_{\mathcal{A}^{(k+1)}}}.$$

This approximates $\hat{\Sigma}_{\mathcal{A}^{(k+1)}}^{-1}\mathbf{1}$ up to a positive scalar, which is what CG seeks. The benefit is limited within a single solve because the outer loop terminates in few outer steps in practice (6–18 on S&P 500 data; see Table 3).

Cross-solve warm-starting (across a sequence of related problems). For an efficient-frontier sweep $\rho_1 < \rho_2 < \cdots < \rho_N$, consecutive optimal portfolios differ by only a few assets entering or leaving the active set. Passing the previous problem's final $(\mathcal{A}, w)$ pair as the initial state for the next solve simultaneously reduces outer iterations (the active set needs fewer primal and dual adjustments) and CG iterations (the initial residual is small). The mechanism is the same for a direct solver such as KKT, but KKT profits only from the warm active set: direct factorisation has no analogue of an initial guess.

The cross-solve warm start is implemented in `solve_cg_warm` (warm active set + CG initial guess) and `solve_kkt_warm` (warm active set only). Section 7 benchmarks `solve_cg_warm` against cold-started CG on a 50-point frontier sweep.

The two strategies make different tradeoffs. The simplex-projection approach is single-stage and loop-free; unlike CG, it is not a Krylov method: each step applies the gradient operator and projects onto Δ_n without building a Krylov subspace, so residuals are not orthogonalised against prior directions, giving $O(\kappa)$ convergence. The active-set loop solves a sequence of linear systems and certifies global optimality via the KKT check; using CG as the inner solver reduces this to $O(\sqrt{\kappa})$ convergence. Both methods share the same $O(nT)$ per-step cost, so the $\sqrt{\kappa}$ gap in iteration

count translates directly to a runtime gap on ill-conditioned problems. Section 6 develops concrete implementations of both strategies.

5 Shrinkage Targets and Preconditioning

5.1 Linear Shrinkage

The sample covariance matrix $\hat{\Sigma} = \frac{1}{T} X^\top X$ is a poor estimator of the true covariance matrix Σ when n is not small relative to T: its smallest eigenvalues are severely downward-biased and the condition number generally far exceeds that of Σ. Ledoit and Wolf [18] propose *linear shrinkage*: replacing $\hat{\Sigma}$ with a convex combination of itself and a symmetric positive definite *shrinkage target* $T_0 = R^\top R$,

$$\hat{\Sigma}_{\mathrm{LW}} = (1-\alpha)\,\hat{\Sigma} + \alpha\, R^\top R,$$

where $\alpha \in (0,1)$ is the shrinkage intensity. Shrinkage moves every sample eigenvalue toward the spectrum of T_0: eigenvalues far from the target shrink more, reducing the condition number of the system matrix. The intensity α can be chosen analytically. The classical choice $R = \sqrt{\bar{\lambda}}\, I$ with $\bar{\lambda} = \|X\|_F^2/(nT)$ gives the scaled-identity target $\bar{\lambda} I$, recovering the original LW estimator, with α set by minimising a consistent estimator of the expected Frobenius loss [18]. The Oracle Approximating Shrinkage (OAS) estimator [4] uses the same target $\bar{\lambda} I$ but a refined formula that achieves lower mean squared error when n/T is non-negligible. For the S&P 500 data ($n/T \approx 0.41$), the two formulas give $\alpha_{\mathrm{LW}} \approx 0.017$ and $\alpha_{\mathrm{OAS}} \approx 0.010$, both small because the Frobenius objective is dominated by the large signal eigenvalues and is largely insensitive to the near-zero noise eigenvalues that drive poor conditioning. Because both oracle estimates use the same target and yield nearly identical solver behaviour, Section 7 reports only the LW oracle alongside $\alpha = 0.5$, representative of the heavier shrinkage levels routinely applied in portfolio construction for out-of-sample stability [6, 17].

Proposition 5.1 (Condition number under linear shrinkage)**.** *Let $\alpha \in (0,1)$, $R^\top R$ SPD. Setting $\hat{\Sigma}_{\mathrm{LW}} = (1-\alpha)\hat{\Sigma} + \alpha R^\top R$, Weyl's inequality [13] gives*

$$\kappa(\hat{\Sigma}_{\mathrm{LW}}) \;\le\; \frac{(1-\alpha)\lambda_{\max}(\hat{\Sigma}) + \alpha\lambda_{\max}(R^\top R)}{\alpha\lambda_{\min}(R^\top R)}\,.$$

The bound is strictly decreasing in α; as $\alpha \to 1$ it approaches $\kappa(R^\top R)$, which equals 1 for $R = \sqrt{\bar{\lambda}}\, I$.

Corollary 5.1 (Scaled-identity target)**.** *For $T_0 = \bar{\lambda} I$ with $\bar{\lambda} = \mathrm{tr}(\hat{\Sigma})/n$, substituting $\lambda_{\max}(T_0) = \lambda_{\min}(T_0) = \bar{\lambda}$ into Proposition 5.1 gives*

$$\kappa(\hat{\Sigma}_{\mathrm{LW}}) \;\le\; \frac{1-\alpha}{\alpha} \cdot \frac{\lambda_{\max}(\hat{\Sigma})}{\bar{\lambda}} + 1\,.$$

The bound is $O\big((1-\alpha)/\alpha\big)$ and depends on the data only through $\lambda_{\max}(\hat{\Sigma})/\bar{\lambda}$; no knowledge of $\lambda_{\min}(\hat{\Sigma})$ is required. The constant $\lambda_{\max}(\hat{\Sigma})/\bar{\lambda}$ is also the condition number of the RMT-clipped target T_0^{RMT} (companion [26]): the rate at which scaled-identity shrinkage improves conditioning is governed by the spectral dominance of the largest factor.

On S&P 500 data the bound is near-tight: at $\alpha = 0.5$ it gives $\kappa(\hat{\Sigma}_{\mathrm{LW}}) \le \lambda_{\max}(\hat{\Sigma})/\bar{\lambda} + 1 \approx 147$, matching the empirical value. The bound saturates because $\lambda_{\min}(\hat{\Sigma}) \approx 6 \times 10^{-7} \ll \alpha\bar{\lambda} \approx 2.3 \times 10^{-4}$, so $\lambda_{\min}(\hat{\Sigma}_{\mathrm{LW}}) \approx \alpha\bar{\lambda}$, which is precisely the denominator assumed by Weyl's inequality.

For CG applied to $\hat{\Sigma}_{\mathrm{LW}}\, v = \mathbf{1}$, Corollary 5.1 makes the α-dependence explicit: the condition number bound is $O((1-\alpha)/\alpha)$, so doubling α from 0.017 to 0.034 roughly halves κ, independent of the noise-eigenvalue floor $\lambda_{\min}(\hat{\Sigma})$. Nonlinear shrinkage [19] applies a different coefficient to each sample eigenvalue, achieving a smaller Frobenius loss when n/T is non-negligible. The resulting estimator is still a fixed SPD matrix, and the matrix-free solvers accept any such target through their dense-target path; what it does *not* admit is the augmented stacked-matrix form $\tilde{X}$, because it is not a scaled identity plus low-rank update of $\hat{\Sigma}$. Its target term therefore costs $O(n^2)$ per application rather than the $O(nk)$ of a factor target – the matrix-free property is retained, only the linear-time target application is lost.

Matrix-free implementation We apply linear shrinkage by augmenting the return matrix rather than forming $\hat{\Sigma}_{\mathrm{LW}}$ explicitly. Setting $c = 1-\alpha$, the stacked matrix

$$\tilde{X} = \begin{pmatrix} \sqrt{c/T}\, X \\ \sqrt{\alpha}\, R \end{pmatrix}$$

satisfies $\tilde{X}^\top \tilde{X} = \frac{c}{T} X^\top X + \alpha R^\top R = \hat{\Sigma}_{\mathrm{LW}}$, so any of the solvers in Section 6 can be applied to $\tilde{X}$ and produce exactly the LW-shrinkage portfolio weights. The augmented-system trick is standard in ridge regression and Tikhonov regularisation [9]; the contributions here are (i) applying it to portfolio optimisation to enable fully matrix-free solvers and (ii) the quantitative identification of LW shrinkage as a preconditioning parameter via the α-dependent bound of Corollary 5.1.

Conditioning as a by-product Heavy LW intensities ($\alpha \approx 0.3$–0.5), already applied by practitioners for out-of-sample stability, also compress κ substantially as a free by-product – a connection not previously made explicit in the portfolio optimisation literature. The mechanism is structural: the LW oracle $\alpha^* = \beta^2/\delta^2$ is small because $\delta^2 = \|\hat{\Sigma} - \bar{\lambda} I\|_F^2$ is dominated by the large, well-estimated eigenvalues (the market factor alone contributes $(\lambda_1 - \bar{\lambda})^2 \approx (0.067)^2$ to δ^2), while each near-zero noise eigenvalue contributes only $\bar{\lambda}^2 \approx 2 \times 10^{-7}$. The near-zero eigenvalues – sole cause of $\kappa \approx 109{,}000$ – receive negligible correction from the oracle; Corollary 5.1 quantifies this: at $\alpha^* \approx 0.017$ the bound gives $\kappa \lesssim 8{,}500$, barely below the unshrunk value. Reaching the practical operating range requires $(1-\alpha)/\alpha = O(1)$, i.e. $\alpha \geq 0.3$, at which point the two objectives are reconciled by the same cross-validation practitioners already perform.

Factor-model targets Practitioners frequently replace the scaled-identity target $\bar{\lambda} I$ with a structured estimate $T_0 = BB^\top + D$, where $B \in \mathbb{R}^{n \times k}$ holds the loadings of $k \ll n$ statistical or commercial risk factors (e.g. principal components of X, Fama-French factors, or Barra model factors) and D is a diagonal idiosyncratic-variance matrix. This target is a drop-in replacement in the stacked-matrix construction: writing

$$R = \begin{pmatrix} B^\top \\ D^{1/2} \end{pmatrix}$$

gives $R^\top R = BB^\top + D$, and $\hat{\Sigma}_{\mathrm{LW}}$ is formed as before without any modification to the solvers. Because the factor structure clusters eigenvalues of $\hat{\Sigma}_{\mathrm{LW}}$ more tightly than a scaled identity, Proposition 5.1 yields a smaller κ bound at the same α, and CG converges in fewer iterations (Proposition 6.1). The matrix-vector product $\hat{\Sigma}_{\mathrm{LW}}\, v$ remains $O(nT)$: the factor and diagonal terms cost $O(nk)$ and $O(n)$, both dominated by the $O(nT)$ data term. The only additional input required is the estimated B and D, a step already present in any factor-model workflow and orthogonal to the computational contribution of this paper.

The stacked-matrix construction extends beyond the scaled-identity family to any SPD shrinkage target T_0.

6 Computational Methods

6.1 CVXPY Reference Implementation

We use CVXPY [7] as a ground-truth reference: the minimum variance objective maps directly to a least-squares norm, which CVXPY dispatches to the Clarabel interior-point solver [10] as a second-order cone program.

Algorithm 3 CVXPY reference solver

Require: Stacked matrix $\tilde{X} \in \mathbb{R}^{(T+n_R)\times n}$; return tilt $\rho\hat{\mu}$
Ensure: Weight vector $w \in \Delta_n$; interior-point iteration count k
1: Declare $w \in \mathbb{R}^n$ as a CVXPY variable
2: Minimise $\|\tilde{X}w\|^2 - \rho\hat{\mu}^\top w$
subject to $\mathbf{1}^\top w = 1,\ w \geq \mathbf{0}$
3: Dispatch to Clarabel interior-point solver [10]
4: **return** $w,\ k$

Interior-point methods require $O(n^3)$ work per iteration for dense problems (as here, where the covariance matrix is fully dense) plus problem-construction overhead. On the S&P 500 problem ($n = 494$) this amounts to roughly 1.9 s (Table 3), making CVXPY unsuitable for production use but ideal as a correctness reference.

All solvers in this paper (CVXPY, CG, and proximal gradient) solve the same optimization problem; on the S&P 500 problem the maximum absolute deviation in portfolio weights between CG and CVXPY (Clarabel) is below 10^{-5}, confirming correctness. The comparison is purely computational. CVXPY and Clarabel are general-purpose tools: the same code handles additional convex constraints (sector limits, turnover bounds, factor-exposure constraints) without modification. The matrix-free methods exploit the specific structure of the pure long-only minimum variance problem and would require rework to accommodate richer constraint sets; practitioners with such requirements may find a specialized QP solver a more appropriate baseline.

6.2 CG Method

The inner solver f for Algorithm 2 is conjugate gradients applied to the $n_a \times n_a$ SPD system derived in Section 3. The action of $\hat{\Sigma}_{\mathrm{LW},a}$ on a vector requires only two matrix-vector products with $\tilde{X}_a$; the matrix itself is never formed.

When an asset is dropped in the primal step, $\mathcal{S}$ is rebuilt for the reduced active set and CG restarts from scratch on the smaller system.

Convergence speed

Proposition 6.1 (CG convergence [11]). *Let $\hat{\Sigma}_a$ be SPD with spectral condition number $\kappa = \kappa(\hat{\Sigma}_{\mathrm{LW},a})$, and let v_k denote the k-th CG iterate for $\hat{\Sigma}_a v = \mathbf{1}$. Then*

$$\frac{\|e_k\|_{\hat{\Sigma}_a}}{\|e_0\|_{\hat{\Sigma}_a}} \leq 2\left(\frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1}\right)^k,$$

Algorithm 4 CG inner step (inner solver f for Algorithm 2)

Require: Active-asset index set active; stacked matrix $\tilde{X}$; return-tilt weight ρ; expected returns $\hat{\mu}$
Ensure: Active-asset weights w_{a}; CG iteration count k

function CgSolve(active)
  Let $\tilde{X}_{\mathrm{a}} \leftarrow \tilde{X}_{:,\mathrm{active}}$, $n_a \leftarrow |\mathrm{active}|$
  Build matrix-free operator $\mathcal{S}$: $v \mapsto \tilde{X}_{\mathrm{a}}^{\top}(\tilde{X}_{\mathrm{a}}\, v)$ *($\hat{\Sigma}_{\mathrm{LW},a}$ matrix-free, $O(n_a T)$)*
  if $\rho = 0$ **then**
    Solve $\mathcal{S}\, v_1 = \mathbf{1}$ via CG
    return $v_1 \,/\, (\mathbf{1}^{\top} v_1)$, k
  else
    Solve $\mathcal{S}\, v_1 = \mathbf{1}$ and $\mathcal{S}\, v_2 = \hat{\mu}_{\mathrm{active}}$ via CG
    $\hat{\lambda} \leftarrow 2(1 - \frac{\rho}{2}\mathbf{1}^{\top} v_2) \,/\, (\mathbf{1}^{\top} v_1)$ *(budget multiplier, exact)*
    return $\frac{\hat{\lambda}}{2} v_1 + \frac{\rho}{2} v_2$, $k_1 + k_2$
  end if
end function

where $\|e\|_{\hat{\Sigma}_a} = (e^{\top}\hat{\Sigma}_a\, e)^{1/2}$ is the $\hat{\Sigma}_a$-energy norm of the error $e = v - v_k$.

The convergence rate is $O(\sqrt{\kappa})$; by Proposition 5.1, Ledoit-Wolf shrinkage compresses κ monotonically in α, directly reducing iteration count. The requirement $\alpha > 0$ in Algorithm 2 ensures that each active-submatrix $\hat{\Sigma}_{\mathrm{LW},\mathcal{A}}$ is strictly positive definite, so Proposition 6.1 applies at every outer step and the inner CG solve is guaranteed to converge to any prescribed tolerance in finitely many iterations.

On the S&P 500 problem this brings CG iterations from 684 (sample covariance, $\alpha = 0$) down to 82 ($\alpha = 0.5$, heavy-shrinkage range [6, 17]); the oracle intensity $\alpha_{\mathrm{LW}} \approx 0.017$ reduces iterations to 516 (one quarter), reflecting the Frobenius objective's insensitivity to the near-zero noise eigenvalues that govern κ.

6.3 Proximal Gradient

Section 4 introduces the simplex-projection strategy and Algorithm 1 gives the matrix-free implementation. Here we analyse its cost and convergence relative to the other methods.

Cost per iteration The gradient is computed as $\frac{1-\alpha}{T} X^{\top}(X w_k) + \alpha\, \hat{\Sigma}_0 w_k$, without assembling $\hat{\Sigma}_{\mathrm{LW}}$ or $\tilde{X}$. The data term costs $O(nT)$; the target term costs $O(n^2)$ for a general $\hat{\Sigma}_0$ or $O(n)$ for a scaled identity. The Lipschitz constant $L = \lambda_{\max}(\hat{\Sigma}_{\mathrm{LW}})$ is estimated by power iteration on the same two-term operator at $O(n_{\mathrm{pow}} \cdot nT)$ setup cost. Each iteration therefore costs $O(nT)$ when $n \ll T$, matching the CG inner step.

Convergence

Proposition 6.2 (Proximal gradient convergence [23])**.** *Let $\hat{\Sigma}_{\mathrm{LW}}$ be SPD, so that $f(w) = \frac{1}{2} w^{\top}\hat{\Sigma}_{\mathrm{LW}}\, w$ is μ-strongly convex and L-smooth on $\mathbb{R}^n$ with $\mu = \lambda_{\min}(\hat{\Sigma}_{\mathrm{LW}}) > 0$ and $L = \lambda_{\max}(\hat{\Sigma}_{\mathrm{LW}})$; both properties therefore hold on $\Delta_n \subset \mathbb{R}^n$. Let $w^{\star} = \arg\min_{w \in \Delta_n} f(w)$. Projected gradient descent with*

step $\tau = 1/L$ satisfies

$$\|w_k - w^\star\|^2 \;\le\; \left(1 - \frac{1}{\kappa(\hat{\Sigma}_{\mathrm{LW}})}\right)^k \|w_0 - w^\star\|^2$$

for all $k \ge 0$.

Proof sketch. Since $w^\star$ minimises f over Δ_n, it is a fixed point: $w^\star = \Pi_{\Delta_n}(w^\star - \tau\nabla f(w^\star))$ for all $\tau > 0$. Non-expansiveness of Π_{Δ_n} and co-coercivity of L-smooth functions ($\|\nabla f(x) - \nabla f(y)\|^2 \le L\langle \nabla f(x) - \nabla f(y),\, x - y\rangle$) give

$$\begin{aligned}\|w_{k+1} - w^\star\|^2 &\le \|(w_k - w^\star) - \tau(\nabla f(w_k) - \nabla f(w^\star))\|^2 \\ &\le \|w_k - w^\star\|^2 - \tau(2 - \tau L)\langle \nabla f(w_k) - \nabla f(w^\star),\, w_k - w^\star\rangle.\end{aligned}$$

Setting $\tau = 1/L$ yields $2 - \tau L = 1$; μ-strong convexity then gives $\langle \nabla f(w_k) - \nabla f(w^\star),\, w_k - w^\star\rangle \ge \mu\|w_k - w^\star\|^2$, so $\|w_{k+1} - w^\star\|^2 \le (1 - \mu/L)\|w_k - w^\star\|^2$. Iterating completes the bound. □

Remark 6.1 (Termination criterion)**.** *Algorithm 1 stops when the step size $\|w_{k+1} - w_k\| \le \varepsilon$. This is a standard surrogate for the first-order optimality condition: at $w^\star$, $w^\star = \Pi_{\Delta_n}(w^\star - \tau\nabla f(w^\star))$, so the step vanishes. The two criteria are equivalent up to a constant: since f is L-smooth, the descent lemma gives $f(w_{k+1}) \le f(w_k) - \frac{L}{2}\|w_{k+1} - w_k\|^2$, so a step of size $\le \varepsilon$ implies an objective decrease of $\le \frac{L}{2}\varepsilon^2$. By Proposition 6.2, $\|w_k - w^\star\|$ decays geometrically at rate $(1 - 1/\kappa)^{1/2}$; non-expansiveness of Π_{Δ_n} then gives $\|w_{k+1} - w_k\| \le \|w_{k+1} - w^\star\| + \|w_k - w^\star\| \le 2\|w_k - w^\star\|$, so the step-size criterion is met as soon as $\|w_k - w^\star\| \le \varepsilon/2$.*

The iteration count therefore scales as $O(\kappa)$, versus $O(\sqrt{\kappa})$ for CG (Proposition 6.1). Since both methods pay $O(nT)$ per step, the $\sqrt{\kappa}$ gap in iteration counts directly translates to a runtime gap. On ill-conditioned problems (the unshrunk S&P 500 system has $\kappa \approx 109{,}000$) this difference is decisive. Proposition 5.1 shows that Ledoit-Wolf shrinkage compresses κ and benefits both methods.

Nesterov acceleration applied to the projected gradient step would improve the rate from $O(\kappa)$ to $O(\sqrt{\kappa})$, matching CG's asymptotic iteration count. However, CG achieves the $O(\sqrt{\kappa})$ rate optimally within the Krylov subspace: each iterate minimises f over $\mathcal{K}_k(\hat{\Sigma}, \mathbf{1})$, which is a strictly richer search space than the two-point momentum used by Nesterov's method. In practice CG typically requires a smaller constant in the $O(\sqrt{\kappa})$ bound. Plain projected gradient is therefore the appropriate first-order comparator: the $\sqrt{\kappa}$ gap between CG and gradient descent is the defining advantage of Krylov methods, and accelerating the gradient baseline would obscure rather than isolate it. For completeness, a FISTA implementation [2] (Beck-Teboulle momentum) is available in the companion package [27]; empirically it requires roughly 1.5–1.8× fewer iterations than plain proximal gradient at the same per-step cost (measured on the S&P 500 benchmark at $\alpha = 0.5$), and remains 7–10× slower than CG, confirming that the advantage over proximal gradient comes from Krylov subspace structure, not Nesterov momentum.

Comparison with other solvers The proximal solver is loop-free and straightforward to implement: it requires no dual feasibility check and no restart logic. Under realistic equity-like spectra (factor-model data, κ growing with n), the scaling experiments (Figure 2) show CG as the fastest solver beyond $n \approx 300$: at $n = 3000$ ($T = 1250$ fixed) CG takes 0.081 s versus 1.53 s for proximal gradient (19× faster). The $O(\kappa)$ iteration count of proximal gradient grows with κ, making it the slowest solver when κ is large. Only when shrinkage is strong enough to compress κ to $O(1)$ does the proximal iteration count compete with CG's $O(\sqrt{\kappa})$; Section 7 discusses this crossover in more detail.

6.4 Complexity Summary

Table 2 collects the asymptotic cost of each method. "Extra memory" is storage beyond the $T \times n$ input matrix X; ε is the solver tolerance; s is the number of active-set outer steps. The two matrix-free methods never form $\hat{\Sigma}$ and require only $O(n)$ working memory beyond X.

Method	Per-step cost	Iterations	Extra memory	Assembles $\hat{\Sigma}$
Clarabel (interior point)	$O(n^3)^{\ddagger}$	$O(\log 1/\varepsilon)$	$O(n^2)$	Yes
OSQP (operator splitting)	$O(n^3)^{\ddagger}$ setup	$O(1/\varepsilon)$	$O(n^2)$	Yes
KKT (dense Cholesky) + active set	$O(k^2T)$	$s^{\dagger}$	$O(k^2)$	Yes
CG matrix-free + active set	$O(kT)$	$s^{\dagger} \cdot O(\sqrt{\kappa})$	$O(n)$	No
Proximal gradient	$O(nT)$	$O(\kappa)$	$O(n)$	No

Table 2: Asymptotic complexity per solver for n assets, T return observations, and condition number $\kappa = \kappa(\hat{\Sigma}_{\mathrm{LW}})$. † s is the number of active-set outer steps (6–18 on S&P 500 data; see Table 3); the CG "Iterations" column counts $s \cdot O(\sqrt{\kappa})$ total inner iterations, where each inner iteration is a pair of $O(kT)$ matrix-vector products. Here $k \leq n$ is the active-set size; the outer loop starts at $k = n$ so the first inner solve dominates, giving effective $O(n^2)$ scaling empirically (Section 7). The reported S&P 500 CG counts (684, 82, 516) are totals summed across all outer steps; because the first solve runs at the full $k = n$ active set and dominates, they are governed by the full-system condition number $\kappa(\hat{\Sigma}_{\mathrm{LW}})$ bounded in Proposition 5.1 and Corollary 5.1, not by the smaller κ of the converged active submatrix. ‡For the direct solvers the per-step column is the dominant cost of a *single factorisation*, not a cheap iteration: Clarabel refactorises an $O(n^3)$ dense KKT system at every interior-point step, whereas OSQP factorises once at $O(n^3)$ setup and then reuses the cached factor in $O(n^2)$ back-solves per ADMM step. The iteration-count entries are the standard worst-case rates ($O(\log 1/\varepsilon)$ for the interior-point method, the ergodic $O(1/\varepsilon)$ for ADMM); both are pessimistic in practice, where Clarabel converges in 11–16 iterations and OSQP in roughly 75–225 (Tables 3–4). The table compares asymptotic structure, not the constant factors that determine wall-clock time. The KKT row assembles the active-set Gram matrix and Cholesky-factorises it ($O(k^2T)$ assembly-dominated, s outer steps, no inner iterations); it is the dense counterpart to matrix-free CG and is faster when the active set is small or the system well-conditioned – the crossover $\kappa < k^2$ – which is why it leads on the 86-asset FTSE problem but trails CG as n grows (Section 7).

7 Numerical Results

All experiments were run on an Apple M4 Pro (14-core CPU, 48 GB RAM) under Python 3.12, NumPy 2.4, SciPy 1.17, CVXPY 1.8.2, and Clarabel 0.11.1. The iterative solvers use SciPy's default relative residual tolerance of 10^{-5}, which was reached in all reported runs; reported times are the fastest of three repeated solves (best-case wall-clock, the usual convention for microbenchmarks as it suppresses operating-system scheduling noise; the relative ordering of solvers is unchanged under mean or median aggregation) and cover only the solve itself; construction of the shrinkage target $\hat{\sigma}^2 I$ is excluded, as it requires a single pass over X and is negligible relative to any solver. We also include a dense baseline, "KKT (Cholesky)": it assembles the active-set Gram matrix $\hat{\Sigma}_{\mathcal{A}}$ explicitly and solves each outer step by a LAPACK Cholesky factorisation (`posv`), wrapped in the same active-set loop as CG. This is the natural production alternative – assemble and factor – and isolates the value of the matrix-free representation, since CG and KKT share the active-set logic and differ only in whether $\hat{\Sigma}$ is formed. It is reported only for the shrunk ($\alpha > 0$) panels: without

shrinkage the assembled matrix can be singular (it is for FTSE), where the dense Cholesky solve is undefined while CG still returns a least-squares iterate. CG is initialised from the zero vector, so its iteration counts are deterministic and exactly reproducible, independent of any random seed. The proximal-gradient comparator instead starts from a random point on the simplex and estimates its Lipschitz step by power iteration, so its iteration count varies mildly across runs (on the S&P 500 problem at $\alpha = 0.5$, 825 ± 87 over ten runs); we report the count from the minimum-time run. All implementations are available in the companion package [27].

7.1 S&P 500 benchmark

The dataset consists of daily percentage returns for 494 S&P 500 constituents over the period July 2021–May 2026 (1213 trading days, $T/n \approx 2.5$), sourced from Yahoo Finance via the `yfinance` Python package. Tickers with more than 5% missing observations are excluded; short gaps are filled by carrying the last observed return forward, which is equivalent to assigning zero return on non-trading days and is the standard convention for inactive market days. The return matrix X is formed by demeaning the returns column-wise.

Spectrum of $\hat{\Sigma} = \frac{1}{T}X^\top X$ The eigenvalue structure of the sample covariance is qualitatively different from that of a random Gaussian matrix and is the reason the S&P 500 problem is a meaningful test case for the preconditioning argument. For a random $T \times n$ matrix with i.i.d. $\mathcal{N}(0, \sigma^2)$ entries, the Marchenko-Pastur law predicts that the bulk of the spectrum lies in the interval $[\sigma^2(1 - \sqrt{n/T})^2,\ \sigma^2(1 + \sqrt{n/T})^2]$, giving a mild condition number that grows modestly with n/T. The equity spectrum is nothing like this.

The largest eigenvalue is $\lambda_1 \approx 0.067$, corresponding to the market factor, which alone accounts for $\approx 30\%$ of total portfolio variance. To apply the Marchenko-Pastur law, we estimate σ^2 as the average squared entry of X,

$$\hat{\sigma}^2 = \frac{\|X\|_F^2}{nT} \approx 0.000457 \ ,$$

the natural entry-level variance of a daily percentage return. The MP upper edge $\hat{\sigma}^2(1 + \sqrt{n/T})^2 \approx 0.00123$ then identifies 23 eigenvalues as statistically significant factors; the remaining 471 are consistent with noise yet still span a wide range down to $\lambda_{\min} \approx 6 \times 10^{-7}$. The resulting condition number is

$$\kappa(\hat{\Sigma}) = \frac{\lambda_{\max}}{\lambda_{\min}} \approx 109{,}000 \ ,$$

far larger than the $O(10)$–$O(100)$ range typical of random matrices at this aspect ratio. The first ten eigenvalues account for half the total variance, while the bottom half of the spectrum contributes less than 3%. This extreme imbalance (a handful of dominant factors and a long tail of near-zero noise eigenvalues) is precisely what makes CG slow without preconditioning (684 iterations) and what makes shrinkage decisive: lifting $\lambda_{\min}$ by even a moderate α dramatically reduces the condition number and thus the iteration count to 82.

Table 3 reports timings and iteration counts on this dataset for the various solvers we consider.

The real-data results reveal two effects. First, without shrinkage the equity system is hard to solve iteratively: CG requires 684 iterations (156× speedup). OSQP (direct API, 0.070 s) is comparable to Clarabel's direct API (0.070 s) and takes 225 iterations; going through CVXPY adds roughly 9× overhead for OSQP (0.61 s) and 26× for Clarabel (1.85 s). Proximal gradient (not a Krylov method) requires ∼6,000 steps at $\kappa \approx 109{,}000$ and is the slowest solver in this panel. The strong factor structure of equity returns concentrates variance along a few eigendirections, inflating the condition number and making first-order iterative methods slow without shrinkage.

Method	Time (s)	Iterations	Speedup
Without shrinkage			
cvxpy (Clarabel)	1.848	11	1.0x
cvxpy (OSQP)	0.610	100	3.0x
Clarabel (direct API)	0.0678	11	27.3x
OSQP (direct API)	0.0705	225	26.2x
CG (SPD)	0.0119	684	155.5x
Proximal gradient†	0.229	6040	8.1x
With LW shrinkage ($\alpha = 0.5$, heavy shrinkage range [6, 17])			
cvxpy (Clarabel)	2.602	16	1.0x
cvxpy (OSQP)	0.579	75	4.5x
Clarabel (direct API)	0.0879	16	29.6x
OSQP (direct API)	0.0380	100	68.4x
KKT (Cholesky)	0.0073	6	358.1x
CG (SPD)	0.0039	82	660.8x
Proximal gradient†	0.0367	769	71.0x
With oracle LW shrinkage ($\alpha_{\text{LW}} \approx 0.017$, [18])			
cvxpy (Clarabel)	1.875	11	1.0x
KKT (Cholesky)	0.0081	15	231.2x
CG (SPD)	0.0114	516	164.9x
Proximal gradient†	0.229	6049	8.2x

Table 3: S&P 500 universe: 494 assets, 1213 trading days (Jul 2021–May 2026). Speedup relative to `cvxpy (Clarabel)` within each panel. The heavy-shrinkage panel uses $\alpha = 0.5$, in the range commonly applied for out-of-sample stability; κ falls to the low hundreds. The oracle LW panel uses $\alpha_{\text{LW}} \approx 0.017$ with target $\bar{\lambda} I$; the more modest gain here reflects the Frobenius objective's insensitivity to near-zero eigenvalues. "Direct API" rows call the same solver without CVXPY's problem-construction layer. CVXPY with LW shrinkage is slower than without (2.60 s vs 1.85 s) because LW requires passing two separate quadratic terms to Clarabel, increasing problem-construction overhead and interior-point iterations from 11 to 16. At $\alpha = 0.5$, CG is 22× faster than Clarabel's direct API (algorithmic gain from exploiting SPD structure matrix-free) and 661× faster than CVXPY-wrapped Clarabel (total, including problem-construction overhead). "Iterations" counts inner solver iterations (CG, Proximal, Clarabel, OSQP). The OAS estimator [4] ($\alpha_{\text{OAS}} \approx 0.010$, same target) gives results nearly identical to the oracle LW panel and is omitted. †Proximal gradient is included as a matrix-free first-order comparator, not a Krylov method.

Second, shrinkage benefits all solvers, and the scale of the benefit is strongly α-dependent. At $\alpha = 0.5$ – in the range of heavier shrinkage levels commonly used in portfolio construction for out-of-sample stability [6, 17] – κ falls to the low hundreds, CG iterations drop to 82, and the speedup grows to 661× (CG). The active-set outer steps also fall, from 18 to 6: shrinkage moves the unconstrained minimum-variance portfolio closer to equal weights, so fewer assets receive negative weight in the primal step and the active set stabilises sooner. OSQP also benefits strongly: iterations fall from 225 to 100 and time from 0.070 s to 0.038 s. The dense KKT (Cholesky) baseline solves the same $\alpha = 0.5$ system in 0.0073 s over 6 outer steps; matrix-free CG is a further 1.9× faster (0.0039 s) and, as Section 7 shows, the margin widens with n. Since the two share the active-set loop and differ only in whether $\hat{\Sigma}$ is assembled, this isolates the not-forming-$\hat{\Sigma}$ representation, rather than the choice of factorisation, as the source of the advantage. Proximal gradient falls from ∼6,000 steps (0.23 s) to ∼800 steps (0.037 s); it remains at least 9× slower than CG because its $O(\kappa)$ count exceeds CG's $O(\sqrt{\kappa})$ even after shrinkage.

The oracle $\alpha_{\text{LW}} \approx 0.017$ gives a more modest gain: CG iterations fall from 684 to 516 (a one-quarter reduction) and the speedup reaches only 165× (CG). This reflects the structural mismatch between the Frobenius objective that produces the oracle and the condition number that governs solver speed: the oracle is largely blind to the near-zero noise eigenvalues that cause ill-conditioning. Note that all iteration counts depend on the factor structure of equity returns and may differ in other market regimes.

7.2 FTSE 100 robustness check

Table 4 repeats the benchmark on FTSE 100 constituents (86 assets, 1698 trading days, Oct 2019–May 2026; daily percentage returns downloaded via `yfinance`, 10% missing-data threshold applied). The FTSE 100 universe has a markedly different spectral character from the S&P 500: with $n/T \approx 0.051$ (vs 0.41 for the S&P 500) the sample covariance is well-estimated and the condition number without shrinkage is $\kappa \approx 516$, roughly 200× lower than the S&P 500 counterpart ($\kappa \approx 109{,}000$). The LW oracle intensity is $\alpha_{\text{LW}} \approx 0.025$, and heavy shrinkage ($\alpha = 0.5$) compresses κ to 27 – an order of magnitude lower than the S&P 500 at the same α.

The speedup pattern mirrors the S&P 500 results of Table 3 despite the different market, time period, and spectral regime. Without shrinkage, CG is 72× faster than CVXPY (870 iterations); at $\alpha = 0.5$ the speedup reaches 560× with only 42 CG iterations. Proximal gradient (22× without shrinkage, 114× at $\alpha = 0.5$) remains consistently slower than CG, as the $O(\kappa)$ vs $O(\sqrt{\kappa})$ gap predicts. One ordering does change: at $\alpha = 0.5$ the dense KKT (Cholesky) baseline is the fastest solver here (711×), narrowly ahead of CG (560×). This is the expected crossover – with only $n = 86$ assets the $O(k^3)$ factorisation is cheap and the condition $\kappa < k^2$ (Section 6) tips in favour of the direct solve; the matrix-free advantage emerges as n grows, as the scaling study (Section 7) confirms. At $n = 86$ the absolute speedups are also dominated by the fixed problem-construction overhead of the CVXPY reference rather than by asset count, so the FTSE panel should be read as confirming the *ordering* of the solvers, not the magnitude of the S&P 500 speedups.

7.3 Out-of-sample reconciliation of the two objectives

Sections 5 and the benchmarks above establish that the *conditioning* of the system improves monotonically in the shrinkage intensity α, whereas the *Frobenius* loss that produces the LW oracle is minimised at a small α (here $\alpha_{\text{LW}} \approx 0.017$). Taken alone, this looks like a trade-off between statistical accuracy and solver speed. It is not. The Frobenius loss is only a surrogate for what an investor actually cares about, the realised out-of-sample variance, and the two are minimised at

Method	Time (s)	Iterations	Speedup
Without shrinkage ($\kappa \approx 516$)			
cvxpy (Clarabel)	0.700	16	1.0x
cvxpy (OSQP)	0.0688	100	10.2x
Clarabel (direct API)	0.0030	16	230.7x
OSQP (direct API)	0.0204	4000	34.3x
CG (SPD)	0.0097	870	71.8x
Proximal gradient†	0.0316	1890	22.2x
With LW shrinkage ($\alpha = 0.5$, $\kappa \approx 27$)			
cvxpy (Clarabel)	0.507	11	1.0x
cvxpy (OSQP)	0.0636	75	8.0x
Clarabel (direct API)	0.0023	11	216.1x
OSQP (direct API)	0.0012	100	408.5x
KKT (Cholesky)	0.0007	4	711.1x
CG (SPD)	0.0009	42	560.1x
Proximal gradient†	0.0045	233	113.8x

Table 4: FTSE 100 universe: 86 assets, 1698 trading days (Oct 2019–May 2026). Speedup relative to `cvxpy (Clarabel)` within each panel. "Iterations" counts inner solver iterations (CG, Proximal, Clarabel, OSQP). With $n/T \approx 0.051$ the sample covariance is well-estimated and κ is already moderate; heavy shrinkage ($\alpha = 0.5$) reduces it further to 27, at which point CG converges in only 42 iterations. The ordering of methods and the qualitative speedup pattern match Table 3 despite the different market and factor structure. †Proximal gradient is included as a matrix-free first-order comparator.

very different intensities.

To show this we run a rolling-window backtest on the S&P 500 universe ($n = 494$). At each rebalance we estimate $\hat{\Sigma}_{\text{LW}}$ on a trailing window of $L = 504$ trading days ($n/L \approx 0.98$, the high-dimensional regime in which shrinkage matters), solve the long-only minimum-variance portfolio, hold it for $H = 21$ trading days, and record the realised daily portfolio returns; rolling forward gives 33 non-overlapping out-of-sample months. The per-window LW oracle intensity averages $\bar{\alpha}_{\text{LW}} = 0.043$ (range 0.019–0.067). We sweep a grid of fixed α and report, for each, annualised out-of-sample volatility, Sharpe ratio, average one-way turnover per rebalance (drift-adjusted), and the mean CG iteration count of the solve (Table 5).

The realised out-of-sample volatility is essentially flat across two orders of magnitude of α, ranging only from 9.66% to 9.78% for $\alpha \in [0.01, 0.5]$, and is *minimised at* $\alpha = 0.30$ (9.66%) – squarely inside the heavy-shrinkage range, far from the Frobenius oracle $\bar{\alpha}_{\text{LW}} \approx 0.04$. Heavy shrinkage $\alpha = 0.5$ delivers 9.74%, statistically indistinguishable from the oracle-scale $\alpha = 0.017$ (9.77%) and marginally better, while requiring only 90 CG iterations against 587 at $\alpha = 0.017$, a 6.5× reduction. Turnover falls monotonically from 14.4% at $\alpha = 0.01$ to 9.1% at $\alpha = 0.5$, so the heavy-shrinkage portfolio is also the cheaper one to trade. Only beyond $\alpha \approx 0.5$ does out-of-sample risk begin to rise as the estimate is pulled too far toward the scaled identity.

The volatility differences across the practical range are not statistically significant, which is the point: a moving-block bootstrap (block length 21, 2000 resamples) gives heavily overlapping 95% confidence intervals ([8.4, 11.8]% at $\alpha = 0.017$ versus [8.1, 12.1]% at $\alpha = 0.5$), and the *paired* difference vol(0.5) − vol(0.017) has bootstrap mean −0.04 percentage points with 95% interval

α	OOS vol (% ann.)	Sharpe	Turnover (%)	CG iters
0.010	9.78	1.533	14.40	678
0.017	9.77	1.532	14.14	587
0.030	9.76	1.536	14.02	483
0.050	9.74	1.536	13.79	404
0.100	9.70	1.530	13.12	293
0.200	9.67	1.484	11.82	185
0.300	9.66	1.441	10.82	137
0.500	9.74	1.374	9.05	90
0.700	9.94	1.253	7.47	64
0.900	10.53	1.062	5.10	40

Table 5: Rolling-window out-of-sample minimum-variance backtest on the S&P 500 ($n = 494$, estimation window $L = 504$ days, monthly rebalance, 33 out-of-sample months). "OOS vol" is the annualised standard deviation of realised daily portfolio returns; turnover is the average one-way drift-adjusted change in weights per rebalance; "CG iters" is the mean inner CG iteration count of the solve at that α. Realised out-of-sample volatility is essentially flat across $\alpha \in [0.01, 0.5]$ and is minimised at $\alpha = 0.30$, inside the heavy-shrinkage range; turnover and CG iterations both fall monotonically in α.

$[-0.40, +0.37]$ straddling zero ($P(\alpha{=}0.5 \text{ worse}) = 0.41$). Heavy shrinkage is thus statistically indistinguishable from the Frobenius oracle out-of-sample, at a fraction of the iteration cost. The conclusion is also robust to the estimation window: repeating the backtest with a shorter $L = 252$ days (the rank-deficient regime $n/L \approx 1.96$) moves the out-of-sample optimum only to $\alpha = 0.20$ – still well inside the heavy-shrinkage range – with $\alpha = 0.5$ and the oracle again indistinguishable (10.5% each).

The picture (Figure 1) is therefore the opposite of a trade-off: the intensity that makes the system well-conditioned and the solver fast is also the intensity that minimises realised out-of-sample risk while lowering turnover. The apparent statistics-versus-speed tension is an artefact of using the in-sample Frobenius loss as the statistical objective; measured against the quantity investors actually optimise, heavy shrinkage and good conditioning coincide. This is the empirical counterpart to the reconciliation argued analytically in Section 5.

7.4 Scaling with problem size

Figure 2 uses synthetic data from the `simulate_equity_returns` generator, whose model is

$$X = FB^\top + E,$$

where $F \in \mathbb{R}^{T\times k}$ are factor returns ($k = \max(3, \lfloor n/10 \rfloor)$), $B \in \mathbb{R}^{n\times k}$ are factor loadings, and E is idiosyncratic noise. The market factor (column 1 of F) has daily volatility 1% and loadings $\beta_i \sim \text{Uniform}(0.4, 0.8)$. The remaining $k-1$ style/industry factors have volatility 0.5% and sparse loadings (Bernoulli$\frac{1}{2}$ mask, $\mathcal{N}(0, 0.04)$ on active entries). Idiosyncratic noise has per-asset daily volatility $\sigma_i \sim \text{Uniform}(0.5\%, 1.5\%)$. This produces a dominant market eigenvalue, secondary factor eigenvalues, and a long noise tail, qualitatively matching real equity spectra.

The condition number grows with n for two reasons. For $n < T = 1250$: the market factor's contribution to $\lambda_{\max}$ grows with n while $\lambda_{\min}$ is pinned by the idiosyncratic floor, widening the spectral spread. For $n > T$: the unshrunk sample covariance becomes rank-deficient (at most T non-zero eigenvalues), so the condition number is infinite without shrinkage; LW shrinkage lifts the

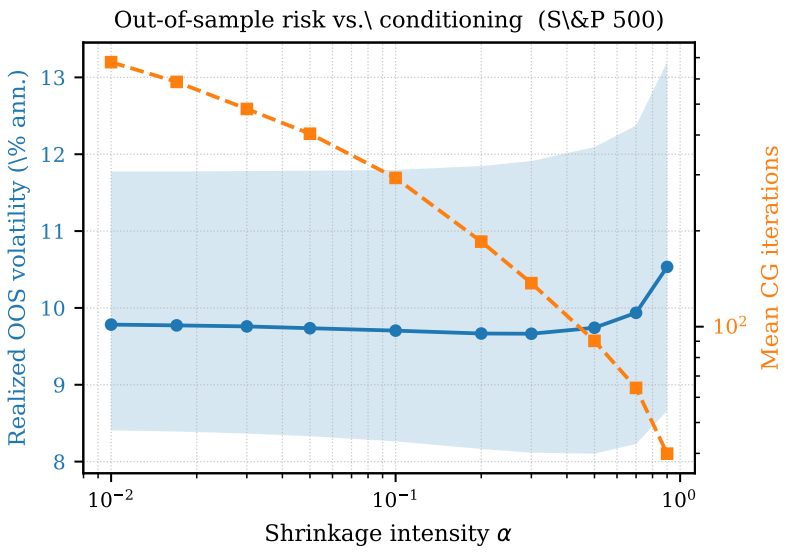

Figure 1: Realised out-of-sample annualised volatility (left axis, solid; shaded band is the 95% moving-block bootstrap confidence interval) and mean CG iteration count (right axis, log scale, dashed) versus shrinkage intensity α for the rolling S&P 500 backtest of Table 5. Out-of-sample risk is minimised inside the heavy-shrinkage range where the iteration count is already small; the two objectives are aligned, not opposed.

zero eigenvalues but the effective κ continues to grow. Both effects are distinct from i.i.d. Gaussian data, which gives $\kappa = O(1)$ at fixed T/n by the Marchenko-Pastur law.

Figure 2 fixes the lookback window at $T = 1250$ (five years of daily returns) and varies n, representing the practical constraint that the estimation window is determined by data availability, not by portfolio size.

CG operates on the k-column submatrix of $\tilde{X}$ at each outer step, where k is the number of assets currently held at positive weight; the first step at $k = n$ dominates the total time. At $\alpha = 0.5$ the active set at convergence holds $k \approx 120$, 200, and 350 assets at $n = 500$, 1000, and 3000 respectively. For fixed $\alpha > 0$, κ_{active} is bounded above by Corollary 5.1, so CG's cost scales asymptotically as $O(nT)$; the observed growth in Figure 2 is steeper than $O(n)$ because κ_{active} increases with n under the factor structure (the dominant market eigenvalue grows while the idiosyncratic floor is fixed), but remains substantially subquadratic; a log-log least-squares fit over the seven grid points with $n \in [300, 3000]$ (single seed, `rng` $= n$) gives an empirical exponent of 1.7 ($R^2 = 0.98$), between linear and quadratic scaling. At $n = 3000$ CG takes 0.081 s, against 0.215 s for the dense KKT (Cholesky) baseline: the two are within 1.9× at $n = 500$ but CG pulls 2.6× ahead by $n = 3000$, the matrix-free margin widening with n exactly as the $O(kT)$-versus-$O(k^2T)$ per-step costs predict.

The decisive separation is between the Krylov solvers and proximal gradient: at $n = 3000$ proximal gradient takes 1.53 s versus 0.081 s for CG, a factor of 19×, reflecting the $O(\kappa)$ versus $O(\sqrt{\kappa})$ iteration-count gap under realistic equity-like spectra.

Figure 3 uses the rank-deficient setting $T = 250 < n = 500$, in which $X^\top X$ has at most T non-zero eigenvalues and the unshrunk sample covariance is singular. This regime is practically relevant: asset universes with more instruments than trading days arise when securities with limited common history are pooled, or when a large cross-section is combined with a short lookback window. Without shrinkage, the KKT system is singular and the iterative solvers do not converge; every point

in the figure has $\alpha > 0$. The preconditioning effect of shrinkage is strongest precisely here: iteration counts fall steeply as α increases from zero, reaching the practical operating point at moderate α values well below unity.

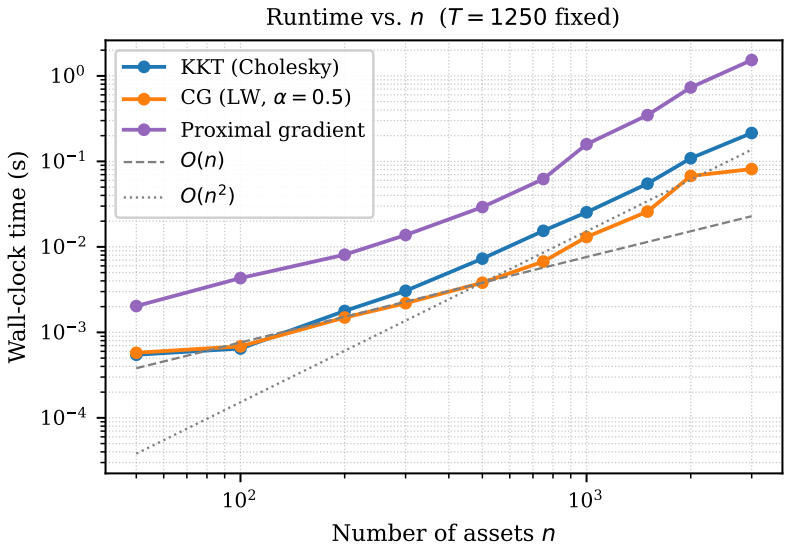

Figure 2: Wall-clock time vs number of assets n (log–log scale, $T = 1250$ fixed) for CG with LW shrinkage ($\alpha = 0.5$), the dense KKT (Cholesky) baseline, and the simplex-projection comparator. Data are generated from a latent factor model. Reference slopes $O(n)$ and $O(n^2)$ are anchored to the CG-LW curve at $n = 500$; CG-LW tracks well below $O(n^2)$ and pulls steadily ahead of the dense KKT baseline as n grows, while the simplex-projection method tracks closer to $O(n^2)$ as its $O(\kappa)$ iteration count grows with the condition number.

7.5 Efficient frontier

The efficient frontier for a long-only mean-variance investor consists of all portfolios that minimise variance for a given expected return. Parameterising by the return-tilt weight $\rho \geq 0$,

$$w^\star(\rho) = \arg\min_{w \in \Delta_n} \; w^\top \hat{\Sigma}_{\mathrm{LW}}\, w - \rho\, \hat{\mu}^\top w\,,$$

sweeping ρ from 0 (pure minimum variance) to $\rho_{\max}$ traces the upper branch of the frontier. Section 3 shows that each solve reduces to one or two SPD systems: $v_1 = \hat{\Sigma}_{\mathrm{LW}}^{-1}\mathbf{1}$ and, when $\rho > 0$, $v_2 = \hat{\Sigma}_{\mathrm{LW}}^{-1}\hat{\mu}$. Without the non-negativity constraint every frontier portfolio is the linear combination $w^\star(\rho) \propto v_1 + \frac{\rho}{2}(v_2 - \langle v_2 \rangle v_1)$, so the unconstrained frontier follows from exactly two CG solves regardless of how many points are requested. With the long-only constraint $w \geq 0$ the active set shifts as ρ changes and each point requires a separate active-set loop. Consecutive solves share a similar active set, so the *warm-start* strategy passes the previous active-set mask and weight vector as the initial state for the next ρ; the CG initial guess is set to the previous solution restricted to the new active set (see Section 4).

Experiment We generate a factor-model return matrix $X \in \mathbb{R}^{1250 \times 500}$ using `simulate_equity_returns` with seed 42. Expected returns are constructed as $\hat{\mu}_i = \beta_i \cdot r_m$, where $\beta_i \sim \mathrm{Uniform}(0.4, 0.8)$ and $r_m = 0.10/250$ is a daily market premium corresponding to a 10% annual return. We use LW

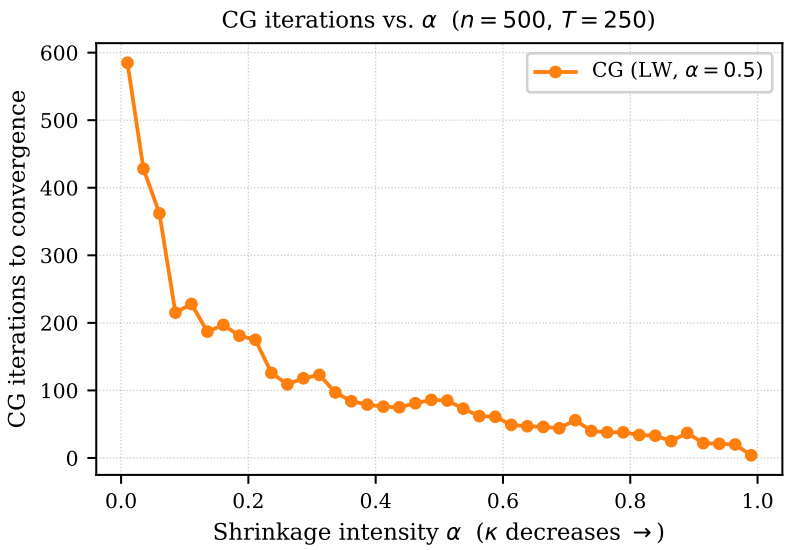

Figure 3: CG iteration counts vs shrinkage intensity α at fixed $n = 500$, $T = 250$ (rank-deficient, factor-model data). Larger α compresses the eigenvalue spectrum and reduces $\kappa(\hat{\Sigma}_{\text{LW}})$, directly reducing the $O(\sqrt{\kappa})$ CG iteration count.

shrinkage at $\alpha = 0.5$ (the heavy-shrinkage panel of Table 3). We compute $N_\rho = 50$ frontier points at $\rho \in [0, 2]$ and record total wall-clock time for each sweep strategy.

Results Figure 4 shows the resulting efficient frontier in annualised risk-return space, with each LW point coloured by the number of active assets. The minimum-variance portfolio under LW shrinkage (orange star, $\rho = 0$) holds 130 active assets at 4.9% annualised volatility; as ρ increases the active set shrinks to ≈ 31 assets at $\rho = 2$. Table 6 reports 50-point sweep times for all solvers.

	Cold start		Warm start	
Solver	Total (s)	ms/point	Total (s)	ms/point
cvxpy (Clarabel)	75.0	1500.3	–	
OSQP (direct API)	2.387	47.7	–	
Proximal gradient	1.492	29.8	–	
CG (LW, $\alpha = 0.5$)	0.239	4.8	0.0515	1.0

Table 6: 50-point efficient frontier sweep ($n = 500$, $T = 1250$, factor-model synthetic data). "Cold start" solves each ρ independently; "warm start" passes the active-set mask and weight vector from the previous ρ point. OSQP and proximal gradient have no warm-start API for the active-set initialisation used here. Timings are for factor-model synthetic data ($n = 500$, $T = 1250$); the S&P 500 universe is comparable in size and gives qualitatively identical results. Cold CG (LW) is 313× faster than CVXPY; warm CG (LW) is 1,455× faster.

Cold-started CG (LW, $\alpha = 0.5$) completes the sweep in 0.239 s (4.8 ms per point); warm-starting reduces this to 0.052 s (1.0 ms per point), a 4.6× improvement. OSQP (2.4 s) and proximal gradient (1.5 s) are respectively 10× and 6× slower than cold CG (LW) because their per-step cost does not decrease as the active set contracts. CVXPY's interior-point loop (75.0 s total, or 1.5 s per point)

makes a naïve loop over CVXPY solves impractical for any application requiring repeated frontier computation.

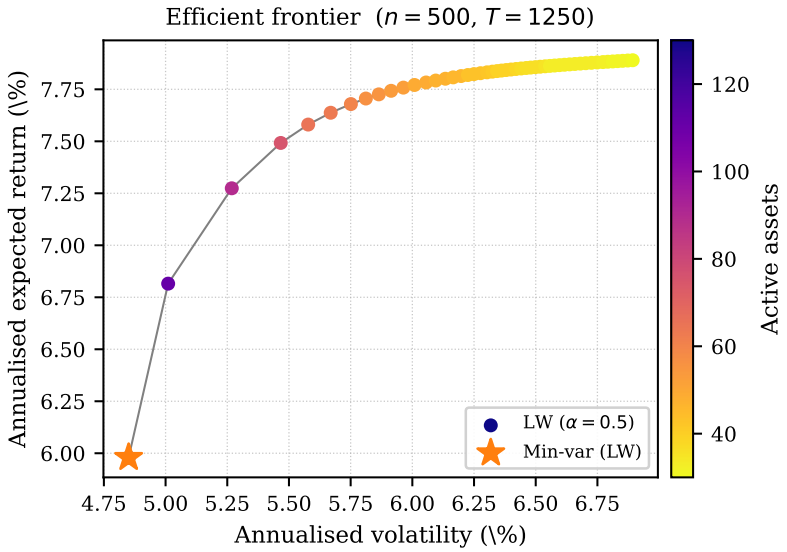

Figure 4: Efficient frontier for $n = 500$ assets and $T = 1250$ daily returns (factor-model synthetic data). Coloured points: LW shrinkage $\alpha = 0.5$; colour encodes active-asset count. The orange star marks the minimum-variance portfolio (warm-started CG). Sweep timings appear in Table 6.

8 Conclusions

The long-only minimum variance portfolio is conventionally solved by forming an $n \times n$ covariance matrix and passing it to a general-purpose solver. This paper shows that the covariance matrix is unnecessary: the problem reduces to one symmetric positive definite linear solve, with non-negativity enforced either by an active-set loop or by projecting gradient steps directly onto the probability simplex. Both strategies work entirely with the $T \times n$ returns matrix; the covariance matrix is never assembled during the solve.

The empirical results on 494 S&P 500 constituents over five years confirm the practical value of this approach. Against the same solvers invoked through their native APIs – the fair comparison, free of any modelling-layer overhead – CG is 5.7× faster than Clarabel without shrinkage and 22× faster under heavy shrinkage ($\alpha = 0.5$); the larger 156× and 661× figures, measured against CVXPY-wrapped Clarabel, additionally absorb the problem-construction overhead of a general-purpose modelling layer and should be read as end-to-end rather than algorithmic gains. At large n, CG's runtime grows substantially subquadratically: at $n = 3000$ (factor-model synthetic data, $T = 1250$ fixed) CG takes 0.081 s, a gap over interior-point methods that widens as n grows (Figure 2). Ledoit-Wolf shrinkage benefits all solvers by compressing the eigenvalue spectrum: it reduces CG iterations from 684 to 82 and OSQP iterations from 225 to 100. The outer active-set loop also contracts from 18 to 6 steps, because a lower condition number moves the unconstrained portfolio toward equal weights, so fewer assets receive negative weights in the primal step. Notably, OSQP called through its direct API (bypassing CVXPY overhead) is still 6–10× slower than CG (ranging from 6× without shrinkage to 10× at $\alpha = 0.5$): even a well-engineered general QP solver cannot match algorithms that exploit the positive-definite structure and operate without assembling the

covariance matrix.

Heavy LW shrinkage compresses κ as a free by-product of out-of-sample stability tuning – a connection not previously made explicit in the portfolio optimisation literature. The statistically optimal $\alpha \approx 0.017$ barely helps conditioning, because the Frobenius loss is dominated by large, well-estimated eigenvalues that are far from zero; the near-zero noise eigenvalues, the sole cause of $\kappa \approx 109{,}000$, contribute negligibly to the loss and receive negligible correction. Practitioners who apply $\alpha \approx 0.3$–0.5 for out-of-sample stability obtain the preconditioning benefit as a free by-product. RMT eigenvalue cleaning [26] resolves the tension more decisively by clipping noise eigenvalues to their theoretically predicted bulk mean, reducing κ to 146 on S&P 500 data; the Woodbury direct solver for the RMT target is developed in the companion paper.

The simplex-projection comparator is 19× slower than CG at $n = 3000$, as its $O(\kappa)$ iteration count grows with the condition number while CG's $O(\sqrt{\kappa})$ count does not. Both strategies extend naturally to the mean-variance problem with a return tilt, which requires solving two SPD systems and recovering the budget multiplier analytically, as outlined in Section 3. The frontier experiment in Section 7 demonstrates this concretely: a 50-point long-only efficient frontier for $n = 500$ assets is completed 1,455× faster than CVXPY (warm-started CG with LW shrinkage, Table 6).

References

[1] I. Bajeux-Besnainou, W. Bandara, and E. Bura, *A Krylov subspace approach to large portfolio optimization*, Journal of Economic Dynamics and Control, 36 (2012), pp. 1688–1699.

[2] A. Beck and M. Teboulle, *A fast iterative shrinkage-thresholding algorithm for linear inverse problems*, SIAM Journal on Imaging Sciences, 2 (2009), pp. 183–202.

[3] J. Bun, J.-P. Bouchaud, and M. Potters, *Cleaning large correlation matrices: tools from random matrix theory*, Reviews of Modern Physics, 89 (2017), p. 035005.

[4] Y. Chen, A. Wiesel, Y. C. Eldar, and A. O. Hero, *Shrinkage algorithms for MMSE covariance estimation*, IEEE Transactions on Signal Processing, 58 (2010), pp. 5016–5029.

[5] L. Condat, *Fast projection onto the simplex and the ℓ_1 ball*, Mathematical Programming, 158 (2016), pp. 575–585.

[6] V. DeMiguel, L. Garlappi, F. J. Nogales, and R. Uppal, *A generalized approach to portfolio optimization: Improving performance by constraining portfolio norms*, Management Science, 55 (2009), pp. 798–812.

[7] S. Diamond and S. Boyd, *CVXPY: A Python-embedded modeling language for convex optimization*, Journal of Machine Learning Research, 17 (2016), pp. 1–5.

[8] J. Duchi, S. Shalev-Shwartz, Y. Singer, and T. Chandra, *Efficient projections onto the ℓ_1-ball for learning in high dimensions*, in Proceedings of the 25th International Conference on Machine Learning (ICML), New York, NY, 2008, ACM, pp. 272–279.

[9] G. H. Golub and C. F. Van Loan, *Matrix Computations*, Johns Hopkins University Press, 4th ed., 2013.

[10] P. J. Goulart and Y. Chen, *Clarabel: An interior-point solver for conic optimisation problems with quadratic objectives*, INFORMS Journal on Computing, 36 (2024), pp. 481–499.

[11] A. Greenbaum, *Iterative Methods for Solving Linear Systems*, vol. 17 of Frontiers in Applied Mathematics, SIAM, Philadelphia, 1997.

[12] M. R. Hestenes and E. Stiefel, *Methods of conjugate gradients for solving linear systems*, Journal of Research of the National Bureau of Standards, 49 (1952), pp. 409–436.

[13] R. A. Horn and C. R. Johnson, *Matrix Analysis*, Cambridge University Press, 2nd ed., 2013.

[14] J. J. Júdice and F. M. Pires, *A block principal pivoting algorithm for large-scale strictly monotone linear complementarity problems*, Computers & Operations Research, 21 (1994), pp. 587–596.

[15] J. Kim and H. Park, *Fast nonnegative matrix factorization: An active-set-like method and comparisons*, SIAM Journal on Scientific Computing, 33 (2011), pp. 3261–3281.

[16] L. Laloux, P. Cizeau, J.-P. Bouchaud, and M. Potters, *Noise dressing of financial correlation matrices*, Physical Review Letters, 83 (1999), pp. 1467–1470.

[17] O. Ledoit and M. Wolf, *Honey, I shrunk the sample covariance matrix*, Journal of Portfolio Management, 30 (2004), pp. 110–119.

[18] O. Ledoit and M. Wolf, *A well-conditioned estimator for large-dimensional covariance matrices*, Journal of Multivariate Analysis, 88 (2004), pp. 365–411.

[19] ——, *Analytical nonlinear shrinkage of large-dimensional covariance matrices*, Annals of Statistics, 48 (2020), pp. 3043–3065.

[20] H. Markowitz, *The optimization of a quadratic function subject to linear constraints*, Naval Research Logistics Quarterly, 3 (1956), pp. 111–133.

[21] R. C. Merton, *An analytic derivation of the efficient portfolio frontier*, Journal of Financial and Quantitative Analysis, 7 (1972), pp. 1851–1872.

[22] K. G. Murty, *Linear Complementarity, Linear and Nonlinear Programming*, vol. 3 of Sigma Series in Applied Mathematics, Heldermann Verlag, Berlin, 1988.

[23] Y. Nesterov, *Introductory Lectures on Convex Optimization: A Basic Course*, Kluwer Academic Publishers, Boston, MA, 2004.

[24] J. Nocedal and S. J. Wright, *Numerical Optimization*, Springer, New York, 2nd ed., 2006.

[25] T. Schmelzer, *cvxcla: Critical line algorithm for portfolio optimisation*. https://github.com/cvxgrp/cvxcla, 2024. MIT License.

[26] ——, *Eigenvalue cleaning and direct solvers for long-only portfolio optimisation*. Companion paper, 2026.

[27] T. Schmelzer, *fast-minimum-variance: Solving minimum variance portfolios fast*. https://github.com/Jebel-Quant/fast_minimum_variance, 2026. MIT License.

[28] B. Stellato, G. Banjac, P. Goulart, A. Bemporad, and S. Boyd, *OSQP: An operator splitting solver for quadratic programs*, Mathematical Programming Computation, 12 (2020), pp. 637–672.